

Feature Maps, Kernels and Regularisation

COMP0078: Supervised Learning

Based on lectures by Mark Herbster
University College London
Department of Computer Science

6 October 2025

*Revision notation: items marked with $\star$ are specifically listed in the revision notes file (`revision25-26-V1.pdf`).
All content in this file is examinable.*

Contents

1	Introduction and Overview	1
1.1	Key Topics	1
1.2	Main Idea	1
2	Review: Inner Product Spaces	2
2.1	Vector Spaces	2
2.2	Normed Spaces	2
2.3	Inner Product Spaces	3
2.4	A More Complicated Example	3
3	Review: Convexity	4
3.1	Checking for Convexity	4
3.2	Why Convexity?	5
4	Basis Functions (Explicit Feature Maps)	5
4.1	Linear Models: Recap	5
4.2	Beyond Linear Models: Polynomials	5
4.3	Polynomial Least Squares	5
4.4	The Polynomial Feature Map	6
4.5	General Basis Functions	6
4.6	Examples of Feature Maps	7
5	Computational Considerations$\star$	8
5.1	Cost of Minimizing Squared Loss	8
5.2	Costs in Big O Notation	8
5.3	When is N Too Large?	8
5.4	A Closer Look	9

6 Ridge Regression and Its Dual Solution	9
6.1 Ill-Posed Problems*	9
6.2 Ridge Regression*	9
6.3 Does Regularisation Help?	10
6.4 Dual Representation*	11

1 Introduction and Overview

This document covers the theory of feature maps, kernels, and regularisation in supervised learning. We explore both explicit feature maps (basis functions) and implicit feature maps (the kernel trick), showing how linear methods can be extended to nonlinear regression through lifting to higher-dimensional spaces.

1.1 Key Topics

- Inner product space review
- Convexity review
- Basis Functions (Explicit Feature Maps)
- Ridge Regression and its dual solution
- Kernel Functions (Implicit Feature Maps)

1.2 Main Idea

We show how a linear method such as least squares may be **lifted** to a (potentially) higher dimensional space to provide a nonlinear regression. A feature map is simply a function that maps the “inputs” into a new space. Thus the original method is now nonlinear in original “inputs” but linear in the “mapped inputs”.

- Explicit feature maps are often known as the **Method of Basis Functions**
- Implicit feature maps are often known as the (reproducing) **“Kernel Trick”**

2 Review: Inner Product Spaces

2.1 Vector Spaces

Definition 2.1 (Vector Space over the Reals*). The triple $(X, +, *)$ defines a **vector space** where X is a set and $+: X \times X \rightarrow X$ is vector addition and $*: \mathbb{R} \times X \rightarrow X$ is scalar multiplication with the abbreviation $ax := a * x$. The following properties hold:

1. $\mathbf{x} + \mathbf{y} = \mathbf{y} + \mathbf{x}$
2. $(\mathbf{x} + \mathbf{y}) + \mathbf{z} = \mathbf{x} + (\mathbf{y} + \mathbf{z})$
3. There exists $\mathbf{0} \in X$ such that for all $\mathbf{x} \in X$ then $\mathbf{x} + \mathbf{0} = \mathbf{x}$
4. $\gamma(\mathbf{x} + \mathbf{y}) = \gamma\mathbf{x} + \gamma\mathbf{y}$

5. $(\gamma + \mu)\mathbf{x} = \gamma\mathbf{x} + \mu\mathbf{x}$
6. $\gamma(\mu\mathbf{x}) = (\gamma\mu)\mathbf{x}$
7. $0\mathbf{x} = \mathbf{0}$ and $1\mathbf{x} = \mathbf{x}$

Note 2.2. 1. $\gamma + \mu$ and $\gamma\mu$ are the usual scalar addition and multiplication over $\mathbb{R}$

2. A vector space can be defined over other fields than the reals, for example arithmetic mod-2, i.e., $X = \{0, 1\}$ and the scalar set is just $\{0, 1\}$.

2.2 Normed Spaces

Definition 2.3 (Norm*). A function $\|\cdot\| : X \rightarrow \mathbb{R}$ is a **norm** on a vector space (Definition 2.1) if

1. $\|\mathbf{x}\| = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$
2. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (Triangle inequality)
3. $\|\gamma\mathbf{x}\| = |\gamma|\|\mathbf{x}\|$

Remark 2.4 (Induced Metric). A norm (Definition 2.3) defines a **metric** (distance) between vectors in the space via

$$d(\mathbf{x}, \mathbf{y}) := \|\mathbf{x} - \mathbf{y}\|$$

Exercise 2.5 (Triangle Inequality for Metric). Check that the triangle inequality holds for the induced metric (Remark 2.4): $d(\mathbf{x}, \mathbf{z}) \leq d(\mathbf{x}, \mathbf{y}) + d(\mathbf{y}, \mathbf{z})$.

Example 2.6 (Common Norms). 1. $\|\mathbf{x}\|_p := (\sum_{i=1}^n |x_i|^p)^{1/p}$ where $\mathbf{x} \in \mathbb{R}^n$ and $p \in [1, \infty)$.

2. $\|\mathbf{x}\|_M := \sqrt{\mathbf{x}^\top M \mathbf{x}}$ where $\mathbf{x} \in \mathbb{R}^n$ and M is an $n \times n$ symmetric positive definite matrix (Definition 2.7).

Definition 2.7 (Positive Definite Matrix). A real-valued symmetric $n \times n$ square matrix M is **positive definite** iff

$$\mathbf{x}^\top M \mathbf{x} > 0 \quad (\forall \mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\})$$

Remark 2.8 (Geometric Interpretation of Norms). A norm generalises the notion of euclidean magnitude $\|\cdot\|_2$. The p -norm plays a particularly important role in machine learning where kernel methods may be seen as a generalisation of the case $p = 2$. The case $p = 1$ is particularly important when learning sparse predictors.

The ∞ -norm is defined as:

$$\|\mathbf{x}\|_\infty := \lim_{p \rightarrow \infty} \|\mathbf{x}\|_p = \max_{i \in [n]} x_i \quad (1)$$

The **level set** of f at α is $\{\mathbf{x} : f(\mathbf{x}) \leq \alpha\}$.

Exercise 2.9 (Manhattan Metric). What metric does $\|\cdot\|_1$ correspond to in $\mathbb{R}^2$?

2.3 Inner Product Spaces

Definition 2.10 (Real Inner Product Space^{*}). The quadruple $(X, +, *, \langle \rangle)$ defines an **inner product space** where $(X, +, *)$ is a vector space (Definition 2.1) and $\langle \rangle : X \times X \rightarrow \mathbb{R}$ is an inner product such that:

1. $\langle \mathbf{x}, \mathbf{x} \rangle = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$ and $\langle \mathbf{x}, \mathbf{x} \rangle \geq 0$
2. $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$ (Symmetry)
3. $\langle \gamma \mathbf{x}, \mathbf{y} \rangle = \gamma \langle \mathbf{x}, \mathbf{y} \rangle$ and $\langle \mathbf{x} + \mathbf{y}, \mathbf{z} \rangle = \langle \mathbf{x}, \mathbf{z} \rangle + \langle \mathbf{y}, \mathbf{z} \rangle$ (Linearity)

Proposition 2.11 (Induced Norm). The inner product (Definition 2.10) induces a norm (Definition 2.3) via $\|\mathbf{x}\| := \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$.

Remark 2.12 (Geometric Interpretation of Inner Product). If $\theta := \cos^{-1} \left(\frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \|\mathbf{y}\|} \right)$ then θ is the angle between the vectors (where the norm is as in Proposition 2.11).

Example 2.13 (Common Inner Products). 1. The Euclidean inner product: $\langle \mathbf{x}, \mathbf{y} \rangle := \sum_{i=1}^n x_i y_i$

2. More generally: $\langle \mathbf{x}, \mathbf{y} \rangle_M := \mathbf{x}^\top M \mathbf{y}$ where $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and M is an $n \times n$ symmetric positive definite matrix (Definition 2.7).

Exercise 2.14 (Weighted Inner Product). Check that Example 2.13(2) defines a valid inner product (Definition 2.10).

Note 2.15 (Inner Product Notation). Many different notations are used for inner product including $\langle \mathbf{x}, \mathbf{y} \rangle = (\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y}$

2.4 A More Complicated Example

Example 2.16 (Sobolev-Type Function Space). Let the space be the set of all functions from $[0, \infty) \rightarrow \mathbb{R}$ with the following three properties:

1. $f(0) = 0$
2. f is absolutely continuous (hence $f(b) - f(a) = \int_a^b f'(x) dx$)
3. $\int_0^\infty [f'(x)]^2 dx < \infty$

Addition '+' is the usual addition of functions, similarly multiplication by a scalar. The inner product is defined as:

$$\langle f, g \rangle := \int_0^\infty f'(x) g'(x) dx \quad (2)$$

Exercise 2.17 (Sobolev Space Verification). Argue that Example 2.16 is an inner product space (Definition 2.10).

Example 2.18 (Example Functions and Their Norms). The following functions illustrate the norm induced by the inner product (2) from Example 2.16:

1. $f(x) = \frac{\sin(x)}{x+1}$; $\|f\| \approx 0.68$
2. $f(x) = \frac{\sin(10x)}{x+1}$; $\|f\| \approx 7.06$
3. $f(x) = 0.1x \mathbb{I}[x \leq 10] + (2 - 0.1x) \mathbb{I}[10 < x \leq 20]$; $\|f\| = \frac{1}{\sqrt{5}}$
4. $f(x) = \sin(x)$; Not in space

Exercise 2.19 (Sine Not in Space). Why is $f(x) = \sin(x)$ not in the space of Example 2.16?

3 Review: Convexity

Definition 3.1 (Convex Function*). A function $f : X \rightarrow \mathbb{R}$ is **convex** iff $\forall p, q \in X$ and $\alpha \in (0, 1)$ we have

$$f(\alpha p + (1 - \alpha)q) \leq \alpha f(p) + (1 - \alpha)f(q) \quad (3)$$

A function f is **concave** if $-f$ is convex. A function is additionally **strictly convex** if we can replace “ $\leq$ ” with “ $<$ ” when $p \neq q$.

Definition 3.2 (Convex Set*). A set X is **convex** if $p, q \in X \Rightarrow (\alpha p + (1 - \alpha)q) \in X$ for $\forall \alpha \in (0, 1)$.

Proposition 3.3 (Properties of Convex Functions). 1. If f and g are convex (Definition 3.1) then $f + g$ is convex.

2. If f is convex and g is affine (linear + a constant) then $f(g(\cdot))$ is convex.

3. Suppose M is a symmetric matrix, then M is a PSD matrix (cf. Definition 2.7) iff $f(\mathbf{x}) = \mathbf{x}^\top M \mathbf{x}$ is convex.

4. A level set of a convex function is convex (a convex set in the sense of Definition 3.2).

Exercise 3.4 (Convexity Properties). Prove the results in Proposition 3.3.

3.1 Checking for Convexity

Proposition 3.5 (Convexity Tests). 1. For $f : (a, b) \rightarrow \mathbb{R}$, if f' is increasing then f is convex.

2. For $f : (a, b) \rightarrow \mathbb{R}$, if $f'' \geq 0$ then f is convex.

3. For $f : X \rightarrow \mathbb{R}$ where $X \subseteq \mathbb{R}^n$ is a convex set (Definition 3.2), if the Hessian of f evaluated at $\mathbf{x}$ denoted $H(\mathbf{x})$ is positive semidefinite for all $\mathbf{x} \in X$, then f is convex.

Example 3.6 (Convexity Examples). 1. $f(x) = x^2$ (Quadratic) — convex by Proposition 3.5(2) since $f'' = 2 > 0$.

2. $f(x) = e^{-x}$ (Exponential decay) — convex since $f'' = e^{-x} > 0$.

3. $f(x) = \tanh(x)^2$ (Neural) — **not convex**.

4. $f(x) = x \log(x) + (1 - x) \log(1 - x)$ (Negative entropy) — convex on $(0, 1)$.

3.2 Why Convexity?

Remark 3.7 (Importance of Convexity in ML). • At the heart of many ML algorithms is an optimisation problem.

- For example, minimize error, minimise regularised error, minimise “energy”, maximise likelihood.
- If the (unconstrained) optimisation problem is convex (Definition 3.1) and there is a minima (or more technically a minimal surface), then gradient-based methods can be applied to smooth problems.
- On the other hand, the existence of “many” minima each associated with a distinct function suggests that for many optimisation approaches there will be a high “variance”.
- **Take-away:** everything else equal, convex objectives in ML are simpler to work with.

4 Basis Functions (Explicit Feature Maps)

4.1 Linear Models: Recap

Definition 4.1 (Linear Model). For $X \subset \mathbb{R}^n$ a vector space (Definition 2.1), we consider **linear models**

$$f : X \rightarrow \mathbb{R} \quad \text{such that} \quad f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} \quad (4)$$

for any $\mathbf{x} \in X$ input vector and $\mathbf{w} \in \mathbb{R}^n$ the parameter vector of f .

4.2 Beyond Linear Models: Polynomials

This is all good and well if $\mathbf{x}$ and y are linearly related. But what if they are **not**?

Polynomials. Let us start with the natural non-linear generalization. Let $X = \mathbb{R}$, we model $f : X \rightarrow \mathbb{R}$ as a polynomial of degree $N \geq 0$:

$$f(x) = a_0 + a_1x + a_2x^2 + \cdots + a_Nx^N \quad (5)$$

where $a_0, \dots, a_N \in \mathbb{R}$ are the coefficients.

4.3 Polynomial Least Squares

Then, learning a polynomial (5) to fit a training dataset (wrt square loss) becomes:

$$\min_{a_0, \dots, a_N} \frac{1}{m} \sum_{i=1}^m (a_0 + a_1x_i + a_2x_i^2 + \cdots + a_Nx_i^N - y_i)^2 \quad (6)$$

(!) But the terms x_i^q for $q = 0, \dots, N$ are “just” numbers!

The optimisation problem is still **linear with respect to the parameters** a_q .

4.4 The Polynomial Feature Map

Definition 4.2 (Polynomial Feature Map^{*}). Define $\phi : X \rightarrow \mathbb{R}^{N+1}$ such that

$$\phi(x) = (1, x, x^2, \dots, x^N)^\top \quad (7)$$

“Rename” the parameters $\mathbf{w} = (w_1, \dots, w_{N+1}) = (a_0, \dots, a_N)$.

Then we can model the polynomial f as:

$$f(x) = \mathbf{w}^\top \phi(x) \quad (8)$$

Namely a **linear model** (cf. Definition 4.1), in the “**embedded**” space $\phi(X)$.

Proposition 4.3 (Polynomial Least Squares Solution). *The training objective (6) using the feature map (Definition 4.2) becomes:*

$$\frac{1}{m} \sum_{i=1}^m (\mathbf{w}^\top \phi(\mathbf{x}_i) - y_i)^2 = \|\Phi \mathbf{w} - \mathbf{y}\|^2 \quad (9)$$

where $\Phi \in \mathbb{R}^{m \times N}$ is the input matrix, with i -row corresponding to the embedded point $\phi(\mathbf{x}_i)$ (taking the place of $X \in \mathbb{R}^{m \times n}$).

The objective is minimized by:

$$\hat{\mathbf{w}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y} \quad (10)$$

Same optimization, but we have replaced $\mathbf{x}$ with $\phi(\mathbf{x})$!

4.5 General Basis Functions

Definition 4.4 (Feature Map^{*}). By a **feature map** we mean a function $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^N$

$$\phi(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_N(\mathbf{x}))^\top, \quad \mathbf{x} \in \mathbb{R}^n \quad (11)$$

- The $\phi_1, \dots, \phi_N$ are called **basis functions**
- Vector $\phi(\mathbf{x})$ is called the **feature vector** and the space $\{\phi(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^n\}$ the **feature space**

Proposition 4.5 (Primal Representation). *The non-linear regression function using a feature map (Definition 4.4) has the **primal representation**:*

$$f(\mathbf{x}) = \sum_{j=1}^N \hat{w}_j \phi_j(\mathbf{x}) \quad (12)$$

where

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \|\Phi \mathbf{w} - \mathbf{y}\|^2 = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y} \quad (13)$$

The last equality holds if $(\Phi^\top \Phi)$ is invertible.

4.6 Examples of Feature Maps

Example 4.6 (Bias Term). We've already seen one example with $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^{n+1}$:

$$\phi(\mathbf{x}) = (\mathbf{x}, 1)^\top$$

In the context of linear regression (Definition 4.1), before application of the feature map we had:

$$\arg \min_{\mathbf{w} \in \mathbb{R}^n} \sum_{i=1}^m (\mathbf{w}^\top \mathbf{x}_i - y_i)^2$$

After the feature map we have:

$$\arg \min_{\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}} \sum_{i=1}^m (\mathbf{w}^\top \mathbf{x}_i + b - y_i)^2$$

thus allowing us to learn a linear fit with a constant offset.

Example 4.7 (XOR Function). Consider the XOR function defined as:

x_1	x_2	$x_1 \text{ XOR } x_2$
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1

Does there exist a linear classifier (Definition 4.1) that fits XOR perfectly? What if we add a bias term (cf. Example 4.6)? Why?

What if instead we first apply the feature map $\phi(\mathbf{x}) := (\mathbf{x}, x_1 x_2)^\top$?

Remark 4.8 (Visualization of the Correlation Feature Map). The distance between two points $\mathbf{x}, \mathbf{t}$ after mapping in feature space (cf. Example 4.7) is:

$$\|\phi(\mathbf{x}) - \phi(\mathbf{t})\|_2 = \sqrt{(x_1 - t_1)^2 + (x_2 - t_2)^2 + (x_1x_2 - t_1t_2)^2} \quad (14)$$

Observe that the unit balls in $\mathbb{R}^2$ with respect to the feature-mapped distances are no longer “norm-like.”

Example 4.9 (Second Order Correlations). More generally, the trick behind XOR (Example 4.7) was to add to the original vector the “correlate” x_1x_2 .

More generally for second order correlations if $\mathbf{x} \in \mathbb{R}^n$ we have:

$$\phi(\mathbf{x}) := (\mathbf{x}, x_1x_1, x_1x_2, \dots, x_1x_n, x_2x_2, x_2x_3, \dots, x_2x_n, \dots, x_nx_n)^\top \quad (15)$$

I.e., $\phi : \mathbb{R}^n \rightarrow \mathbb{R}^{\frac{n^2+3n}{2}}$.

Exercise 4.10 (Motivation for Correlations). What is the motivation for the feature map in Example 4.9?

More generally we might also include higher order correlations. What is a potential problem with this technique?

5 Computational Considerations*

5.1 Cost of Minimizing Squared Loss

We have a method for solving the squared loss (Proposition 4.5).

Question. How efficient is it?

In other words, how much time (and memory) would it take to obtain $\hat{\mathbf{w}}$?

Answer. It depends:

- On the machine
- On the training size m
- On the number n of input dimensions

5.2 Costs in Big O Notation

Question. What is the cost in Big O notation?

Recalling the primal solutions (13):

$$\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y} \quad (\text{w/o feature map}) \quad (16)$$

$$\hat{\mathbf{w}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{y} \quad (\text{with feature map } \Phi) \quad (17)$$

Answer:

- $O(n^3 + mn^2)$ in time without feature map (we are referring to standard methods implemented in BLAS/LAPACK)
- $O(N^3 + mN^2)$ in time with feature map

Definition 5.1 (Big O Notation). Informally, $f(x) = O(g(x))$ if it behaves “essentially” like $g(x)$. Formally, there exists $C > 0$ and x_0 such that for any $x > x_0$:

$$|f(x)| < Cg(x) \quad (18)$$

Note 5.2 (Big O Caveats). Two functions with “same” big O behavior (Definition 5.1) might differ a lot by:

- The “activating” x_0 (one might behave like $g(x)$ a lot sooner)
- The constant C (one might have a much larger constant)

Hence two methods with same big O might still be quite different in practice in terms of time and memory.

5.3 When is N Too Large?

Let assume m is fixed, say $m = 1000$. How large can we take N ?

Ideally, as large as possible (well... not really because of overfitting, but we would not like computations to be our bottleneck)!

In practice, not too much: cost grows cubically in N .

Example 5.3 (Polynomial Feature Dimensions). Polynomials $f(x)$ of degree d over n variables ($X = \mathbb{R}^n$):

- $n = 1, d = 3 \Rightarrow N = 4$
- $n = 3, d = 5 \Rightarrow N = 56$
- $n = 5, d = 10 \Rightarrow N = 3003$
- $n = 10, d = 15 \Rightarrow N = 184756$

5.4 A Closer Look

Where is the cost $O(N^3 + mN^2)$ coming from in (17)?

Is there any way we could estimate $\hat{\mathbf{w}}$ faster?

6 Ridge Regression and Its Dual Solution

6.1 Ill-Posed Problems*

Definition 6.1 (Well-Posed Problem (Hadamard, 1902)*). A problem is **well-posed** if:

1. A solution exists
2. The solution is unique
3. The solution depends continuously on the data

A problem is **ill-posed** if it is not well-posed.

Remark 6.2 (Ill-Posedness of Learning). Learning problems are in general ill-posed (Definition 6.1), usually because of condition (2).

Regularization theory provides a general framework to solve ill-posed problems.

Example 6.3 (Linear Interpolation Problem). We wish to find a function $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ (cf. Definition 4.1) which best interpolates a data set $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\} \subseteq \mathbb{R}^n \times \mathbb{R}$.

- If the data have been generated in the form $(\mathbf{x}, f(\mathbf{x}))$, the vectors $\mathbf{x}_i$ are linearly independent and $m = n$, then there is a unique interpolant whose parameter $\mathbf{w}$ solves:

$$X\mathbf{w} = \mathbf{y} \quad (19)$$

where $\mathbf{y} = (y_1, \dots, y_m)^\top$ and $X = [\mathbf{x}_1, \dots, \mathbf{x}_m]^\top$

- Otherwise, this problem is ill-posed (Definition 6.1)

6.2 Ridge Regression*

Motivation:

1. Given a set of k hypothesis classes $\{\mathcal{H}_r\}_{r \in \mathbb{N}_k}$ we can choose an appropriate hypothesis class with cross-validation
2. An alternative compatible with linear regression is to choose a single “complex” hypothesis class and then modify the error function by adding a “complexity” term which penalises complex functions
3. This is known as **regularization**
4. Cross-validation may still be needed to set the regularization parameter (see below) and other parameters defining the complexity term

Definition 6.4 (Ridge Regression*). We minimize the **regularized (penalized) empirical error**:

$$E_{\text{emp}}^\lambda(\mathbf{w}) := \sum_{i=1}^m (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \lambda \sum_{\ell=1}^n w_\ell^2 \equiv (\mathbf{y} - X\mathbf{w})^\top (\mathbf{y} - X\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (20)$$

The positive parameter λ defines a trade-off between the error on the data and the norm (Definition 2.3) of the vector $\mathbf{w}$ (degree of regularization).

Theorem 6.5 (Ridge Regression Solution*). Setting $\nabla E_{\text{emp}}^\lambda(\mathbf{w}) = 0$ in (20), we obtain the **modified normal equations**:

$$-2X^\top (\mathbf{y} - X\mathbf{w}) + 2\lambda \mathbf{w} = 0 \quad (21)$$

whose solution (called **regularized solution**) is:

$$\hat{\mathbf{w}}_\lambda = (X^\top X + \lambda I_n)^{-1} X^\top \mathbf{y} \quad (22)$$

With a feature map ϕ (Definition 4.4):

$$\hat{\mathbf{w}}_\lambda = (\Phi^\top \Phi + \lambda I_N)^{-1} \Phi^\top \mathbf{y} \quad (23)$$

Proof. Setting the gradient of (20) to zero:

$$\nabla_{\mathbf{w}} E_{\text{emp}}^\lambda(\mathbf{w}) = -2X^\top (\mathbf{y} - X\mathbf{w}) + 2\lambda \mathbf{w} = 0$$

Rearranging:

$$X^\top X \mathbf{w} + \lambda \mathbf{w} = X^\top \mathbf{y} \implies (X^\top X + \lambda I_n) \mathbf{w} = X^\top \mathbf{y}$$

Since $\lambda > 0$, the matrix $(X^\top X + \lambda I_n)$ is always invertible, giving (22). The feature map version (23) follows by replacing X with Φ . $\square$

6.3 Does Regularisation Help?

Remark 6.6 (Regularisation vs. Empirical Error). By modifying $\hat{\mathbf{w}}$ into $\hat{\mathbf{w}}_\lambda$ (Theorem 6.5) we are not fitting our data as well:

$$\frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{w}}^\top \phi(\mathbf{x}_i) - y_i)^2 \leq \frac{1}{m} \sum_{i=1}^m (\hat{\mathbf{w}}_\lambda^\top \phi(\mathbf{x}_i) - y_i)^2$$

Is this a problem? Maybe not. We actually care about the **expected risk**:

$$E(\hat{\mathbf{w}}_\lambda) = \mathbb{E}_{(\mathbf{x}, y)} [(\hat{\mathbf{w}}_\lambda^\top \phi(\mathbf{x}) - y)^2]$$

Therefore (in principle at least) it might even happen that $E(\hat{\mathbf{w}}_\lambda) \leq E(\hat{\mathbf{w}})$.

Remark 6.7 (Bias–Variance Decomposition). Regularization has some pros and cons. Let S be a training set:

$$\mathbb{E}_S[E(\hat{\mathbf{w}}_\lambda) - E(f^*)] = \text{Bias}(\hat{\mathbf{w}}_\lambda, f^*) + \text{Variance}(\hat{\mathbf{w}}_\lambda) \quad (24)$$

- **Bias.** “Average” distance from the ground truth:

$$\text{Bias}(\hat{\mathbf{w}}_\lambda, f^*) = \mathbb{E}_{\mathbf{x}} \left[(f^*(\mathbf{x}) - \mathbb{E}_S[\hat{\mathbf{w}}_\lambda]^\top \phi(\mathbf{x}))^2 \right]$$

Hence, if $f^*(\mathbf{x}) = \hat{\mathbf{w}}^\top \phi(\mathbf{x})$, the larger λ the further away our predictions will be... BAD!

- **Variance.** Variance of model’s output w.r.t. training set S :

$$\text{Variance}(\hat{\mathbf{w}}_\lambda) = \mathbb{E}_{\mathbf{x}, S} \left[|(\hat{\mathbf{w}}_\lambda - \mathbb{E}_S[\hat{\mathbf{w}}_\lambda])^\top \phi(\mathbf{x})|^2 \right]$$

The larger λ the smaller the variance (why?)... GOOD!

6.4 Dual Representation*

Theorem 6.8 (Dual Representation of Ridge Regression*). *The regularized solution (Theorem 6.5) can be written as*

$$\mathbf{w} = \sum_{i=1}^m \alpha_i \mathbf{x}_i \quad \Rightarrow \quad f(\mathbf{x}) = \sum_{i=1}^m \alpha_i \mathbf{x}_i^\top \mathbf{x} \quad (25)$$

where the vector of parameters $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_m)^\top$ is given by

$$\boldsymbol{\alpha} = (X X^\top + \lambda I_m)^{-1} \mathbf{y} \quad (26)$$

We call $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ the **primal form** and (25) the **dual form** (or representation). The dual form is computationally convenient when $n > m$.